DSAA 4040 Course Project Handout

Cloud Computing & Big Data Systems

Instructor: Guoming Tang

Spring 2026 | HKUST(GZ)

Updated on Apr. 9, 2026

1. Project Overview

The course project is a group project designed to help you apply key ideas from cloud computing and big data systems in a concrete setting.

Each group may choose one project from one of the following two tracks:

Research-Oriented Projects

Engineering-Oriented Projects

Note: You are allowed to design and propose your own project topic, but it needs the confirmation of instructor.

The two tracks have different goals:

Research-oriented projects focus on asking a clear systems question, designing experiments, collecting evidence, and drawing conclusions.

Engineering-oriented projects focus on designing, implementing, deploying, and demonstrating a working system.

You should choose the track that best matches your team's interests and strengths.

2. Team Size

Recommended team size: 1–2 students per group (recommended); groups of 3 students are allowed with justification.

3. Recommended Environments

To keep the project focused on systems understanding rather than heavy infrastructure setup, lightweight environments are fully acceptable.

For Kubernetes-based projects

You may use:

Minikube

Docker Desktop Kubernetes

K3s

For Spark-based projects

You may use:

PySpark in local mode

Single-node Spark standalone mode

Note: You will not receive extra credit simply for using a more complicated environment.

4. Deliverables

Each group must submit the following:

Source code / configuration files
Project report (PDF)
Presentation slides/Demo video
README with setup and reproduction instructions

5. Report Format

For Research-Oriented Projects

Your report should be written in a mini-paper style and include:

Introduction
Research Question / Hypothesis
Methodology / Experimental Setup
Results
Discussion
Conclusion
References

For Engineering-Oriented Projects

Your report should be written in a technical report style and include:

Problem Statement
System Design
Implementation
Deployment / Demo
Evaluation or Testing
Limitations and Future Work
References

6. Task Levels

Each project contains three levels of requirements:

Basic

Minimum requirements. Every group must complete these.

Standard

Expected level for a strong project.

Advanced / Bonus

Optional challenges for stronger groups. These help distinguish top projects, but quality matters more than quantity.

Note: Completing more bonus tasks does not automatically guarantee a higher grade.

A smaller project done well is better than a larger project done poorly.

Part A. Research-Oriented Projects

R1. A Comparative Study of Auto-Scaling Policies for Cloud-Native Web Applications on Kubernetes

Goal

Study how different scaling strategies behave under different traffic patterns for a cloud-native web application deployed on Kubernetes.

Suggested Environment

Minikube, Docker Desktop Kubernetes, or K3s

A simple web service, such as Flask, FastAPI, or Node.js

Metrics Server, with optional Prometheus/Grafana

Background

Cloud-native systems often face changing workloads. Kubernetes provides mechanisms such as the Horizontal Pod Autoscaler (HPA) to adapt system capacity automatically. This project studies whether auto-scaling actually improves performance and efficiency under different traffic patterns.

Basic Requirements

Deploy a simple web application on Kubernetes.

Compare two scaling strategies:

- fixed number of replicas
- HPA

Design two traffic patterns:

- steady traffic
- burst traffic

Measure at least two metrics, such as:

- response time
- CPU usage
- number of pods over time

Write a clear conclusion on when one strategy works better than the other.

Standard Requirements

Add a third strategy, such as:

- manual scaling
- another fixed replica configuration

Add a third traffic pattern, such as flash crowd or sudden spike.

Run multiple trials and visualize the results.

Analyze trade-offs among:

- latency
- resource usage
- scaling responsiveness

Advanced

Compare different HPA target values.

Analyze scaling delay, instability, or oscillation.

Include a simple cost proxy, such as pod-hours.

Discuss practical limitations of HPA in real systems.

Expected Research Angle

Does auto-scaling always outperform fixed deployment? Under what workload patterns is HPA most useful?

R2. Measuring the Impact of Data Formats on Spark Analytics Performance: CSV, JSON, and Parquet

Goal

Investigate how different storage formats affect the performance of Spark analytics tasks.

Suggested Environment

- PySpark in local mode
- One public or self-generated dataset
- The same dataset stored in CSV, JSON, and Parquet

Background

In data systems, storage format can significantly affect both storage efficiency and query performance. This project explores how row-oriented text formats differ from columnar formats in practical Spark workloads.

Basic Requirements

Prepare the same dataset in three formats:

- CSV
- JSON
- Parquet

Design at least two Spark workloads, for example:

- filter + projection
- aggregation

Compare:

- file size
- read/query time

Summarize the observed differences.

Standard Requirements

Add a third workload, such as:

- groupBy
- join
- selective query

Repeat each experiment multiple times and report average results.

Explain why Parquet often performs better.

Discuss whether the advantage is consistent across workload types.

Advanced

Study schema inference overhead.

Compare compressed vs. uncompressed settings.

Study the effect of dataset size.

Connect your findings to data lake / lakehouse design ideas.

Expected Research Angle

How strongly does storage format affect analytical performance, and under what query patterns are the differences most visible?

R3. Resource Allocation and Fairness in a Shared Kubernetes Cluster

Goal

Study how Kubernetes resource management mechanisms affect performance, interference, and fairness among competing workloads in a shared cluster.

Suggested Environment

- Minikube or K3s
- Metrics Server enabled

Simple workloads such as:

- a CPU-intensive batch job
- a latency-sensitive web server

Background

In a shared Kubernetes cluster, multiple applications may compete for the same CPU and memory resources. Kubernetes provides several mechanisms to control and prioritize resource usage, including:

- Requests / Limits
- PriorityClasses
- ResourceQuota

This project investigates how these mechanisms influence application performance and fairness under contention.

Basic Requirements

Deploy at least two competing workloads in the same cluster, for example:

- one CPU-heavy batch job
- one latency-sensitive web service

Measure system behavior without explicit resource controls.

Apply resource requests and limits to the workloads.

Measure and compare the difference in:

- response time or service quality of the web application
- CPU/memory usage
- job completion behavior

Standard Requirements

Compare at least two Kubernetes mechanisms, such as:

- Requests / Limits
- PriorityClasses
- ResourceQuota

Run experiments under multiple workload intensities.

Analyze whether resource controls improve:

- predictability
- fairness

isolation

Present your results with clear figures and tables.

Advanced

Study how PriorityClasses affect preemption or service quality under stress.

Add a multi-team scenario with namespace-level quotas.

Define and justify a simple fairness metric.

Discuss design implications for shared teaching or research clusters.

Expected Research Angle

Do Kubernetes resource controls meaningfully improve fairness and performance isolation in a shared cluster? Which mechanisms are most useful for different workload types?

Part B. Engineering-Oriented Projects

E1. Design and Deploy a Cloud-Native Online Bookstore on Kubernetes

Goal

Design and implement a small cloud-native online bookstore and deploy it on Kubernetes.

Suggested Environment

- Minikube, Docker Desktop Kubernetes, or K3s
- Frontend + backend + database
- Docker for containerization

Background

Modern cloud applications are typically packaged as containers, deployed on Kubernetes, and configured through declarative manifests. This project asks you to build a small but complete cloud-native application.

Basic Requirements

Implement a minimum working system with:

- a frontend page
- a backend API
- a database

Containerize the application with Docker.

Deploy it on Kubernetes using:

- Deployment
- Service

Support at least two core features, such as:

- browse/search books
- shopping cart
- place order

Standard Requirements

Use ConfigMap and Secret appropriately.

Expose the application through Ingress.

Provide a clean architecture diagram.

Demonstrate a complete deployment workflow.

Advanced

Add auto-scaling.

Add basic monitoring or dashboard support.

Improve the database schema and API design.

Conduct a small performance test.

Expected Engineering Focus

Cloud-native packaging, deployment, service exposure, and configuration management.

E2. Build a Mini Streaming Log Analytics Pipeline with Spark and Parquet Storage

Goal

Build a small streaming log analytics pipeline that ingests logs, cleans them, and stores processed results in Parquet format.

Suggested Environment

- Spark Structured Streaming (Local Mode)
- Parquet storage

Log ingestion via one of the following:

- TCP Socket
- Directory Stream
- Kafka via Docker (optional but encouraged)

Background

Modern data systems often process logs continuously rather than in large offline batches. This project asks you to build a lightweight version of such a pipeline using Spark Structured Streaming.

Basic Requirements

Build a log generator that produces synthetic access logs or event logs.

Ingest logs using one of the following sources:

- TCP Socket
- Directory Stream
- Kafka

Use Spark to:

- parse and clean the logs
- handle invalid or incomplete records reasonably

Save the processed output to Parquet.

Show at least one useful analytics result, such as:

- request count
- error count
- top URLs
- events over time

Standard Requirements

Include multiple log fields, such as:

- timestamp
- user ID
- URL
- status code

Add at least two analytics tasks.

Provide a clear architecture diagram of the pipeline.

Demonstrate the pipeline running continuously on streamed input.

Advanced

Use Kafka instead of a simpler source.

Add a small dashboard or notebook for result visualization.

Compare raw output with cleaned/structured output.

Explore checkpointing, output modes, or simple fault handling.

Expected Engineering Focus

Streaming ingestion, structured processing, Parquet-based storage, and end-to-end pipeline integration.

E3. Design and Implement a Multi-Tenant Kubernetes Lab Platform with RBAC and Resource Isolation

Goal

Design a small Kubernetes-based lab platform for multiple users or teams, with access control and basic resource isolation.

Suggested Environment

- Minikube or K3s
- Simulated users/roles are acceptable
- No enterprise authentication system is required

Background

Shared cloud platforms require basic tenant isolation and permission control. This project asks you to design a simplified multi-tenant Kubernetes environment suitable for a teaching lab or student project platform.

Basic Requirements

Define at least three user roles, such as:

- admin
- developer
- viewer

Configure:

- namespaces
- Role / RoleBinding or ClusterRole / ClusterRoleBinding

Demonstrate different access permissions for different roles.

Provide documentation for how the platform is organized.

Standard Requirements

Add ResourceQuota or LimitRange.

Design the platform as if each student team had its own namespace.

Include a simple onboarding or usage guide.

Explain your security and isolation design clearly.

Advanced

Demonstrate how your design prevents misuse or accidental damage.

Add NetworkPolicy or finer-grained permissions.

Build simple automation scripts or a lightweight portal.

Discuss scalability and limitations of your design.

Expected Engineering Focus

Multi-tenancy, isolation, RBAC, and platform-style Kubernetes design.

7. Project Difficulty Guide

To help you choose an appropriate project, we provide the following difficulty classification:

Level 1 (Moderate)

R2: Data Format vs Spark Performance

Level 2 (Moderate–Challenging)

R1: Auto-Scaling on Kubernetes

E1: Cloud-Native Online Bookstore

E2: Streaming Log Analytics Pipeline

Level 3 (Challenging)

R3: Resource Allocation and Fairness in Kubernetes

E3: Multi-Tenant Kubernetes Platform

Recommendation

If your goal is to complete a solid and stable project, choose a Level 1–2 project.

If your goal is to aim for top grades (A/A+), consider a Level 3 project, but be prepared for higher uncertainty.

8. Grading Rubric

Research-Oriented Projects

Problem formulation: 15%

Methodology / experiment design: 25%

Implementation correctness: 20%

Analysis and insight: 25%

Proposal/Presentation/Report&Demo: 15%

Engineering-Oriented Projects

System design: 20%

Implementation quality: 25%

Deployment / demo: 20%

Completeness and robustness: 20%

Proposal/Presentation/Report&Demo: 15%

9. Milestones

Milestone 1: Topic Selection (Due: Apr. 15, 2026)

Each group must submit:

- project title
- team members
- selected track
- short proposal (1 page)

Milestone 2: Progress Demonstration (May. 7, 2026)

Each group must present their progress (8~10 minutes each group):

- presentation
- demo video / live demo (optional)

Milestone 3: Final Submission (May. 21, 2026)

Each group must submit:

- code/configuration
- report
- demo video

10. Academic Integrity and Use of AI Tools

You may use AI tools such as ChatGPT, Claude, or Copilot to help with:

- brainstorming,
- debugging,
- explaining errors,
- writing boilerplate code,

polishing documentation.

However:

You must understand and verify all submitted work.

You must disclose major AI usage in an appendix or short note.

Blindly copying AI-generated code or text without verification is not acceptable.

Your project will be evaluated based on your understanding, not on how much code you can generate.

11. Project Advice

Choose a topic that matches your team’s strength.

Choose a research-oriented project if your team enjoys experiments, comparisons, and analysis.

Choose an engineering-oriented project if your team enjoys building and demonstrating working systems.

A well-scoped, clearly executed project is better than an over-ambitious one.

Good luck :)